Introduction to Theoretical Computer Science

CS2490 - Session 2: Alphabets, Words, and Languages

Pierre Cagne

Fall 2026

August 19, 2026

Alphabet

Definition

An *alphabet* is a finite (non-empty) set of symbols.

Alphabet

Definition
An *alphabet* is a finite (non-empty) set of symbols.

When you decide on an alphabet, you are deciding which symbols your input data can contain.

Alphabet

Definition
An *alphabet* is a finite (non-empty) set of symbols.

When you decide on an alphabet, you are deciding which symbols your input data can contain.

Although everything can be encoded in binary ultimately, it is often more convenient to work with a specific alphabet that matches the data you are dealing with.

Examples of Alphabets

Examples:

Examples of Alphabets

Examples:

- The English alphabet $A = \{a, b, c, \ldots, z\}$ (lower-case only).

Examples of Alphabets

Examples:

- The English alphabet $A = \{a, b, c, \ldots, z\}$ (lower-case only).
- The Binary alphabet $B = \{0, 1\}$.

Examples of Alphabets

Examples:

- The English alphabet $A = \{a, b, c, \ldots, z\}$ (lower-case only).
- The Binary alphabet $B = \{0, 1\}$.
- The Calculator alphabet $C = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, +, -, \times, \div, =, ., \%\}$.

Examples of Alphabets

Examples:

- The English alphabet $A = \{a, b, c, \ldots, z\}$ (lower-case only).
- The Binary alphabet $B = \{0, 1\}$.
- The Calculator alphabet $C = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, +, -, \times, \div, =, ., \%\}$.
- The DNA alphabet $D = \{A, C, G, T\}$.

Examples of Alphabets

Examples:

- The English alphabet $A = \{a, b, c, \ldots, z\}$ (lower-case only).
- The Binary alphabet $B = \{0, 1\}$.
- The Calculator alphabet $C = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, +, -, \times, \div, =, ., \%\}$.
- The DNA alphabet $D = \{A, C, G, T\}$.
- The Java alphabet

$$J = \{\texttt{abstract}, \texttt{boolean}, \texttt{break}, \texttt{case}, \texttt{catch}, \texttt{char}, \texttt{class}, \texttt{else}, \texttt{enum}, \ldots\} \cup \{a, \ldots, z, A, \ldots, Z\} \cup \{0, 1, \ldots, 9\} \cup \{., ;, (,), \{, \}, ...\} \cup \ldots$$

Notation for alphabets

Everything we will see this semester will work for any alphabet. We will only fix alphabets in examples.

Notation for alphabets

Everything we will see this semester will work for any alphabet. We will only fix alphabets in examples.

We will use upper-case Greek letters ($\Sigma, \Gamma, \ldots$) to denote an arbitrary alphabet. The symbols of an alphabet are called *letters*.

Word

Definition

Let Σ be an alphabet. A *word* over Σ is a finite sequence of letters from Σ.

Word

Definition

Let Σ be an alphabet. A *word* over Σ is a finite sequence of letters from Σ.

A word is simply the input data on which you want to run your machine.

Word

Examples:

Word

Examples:

- 1101100110101100001 is a word over the Binary alphabet B.

Word

Examples:

- 1101100110101101100 01 is a word over the Binary alphabet B.
- The word *english* is a word over the English alphabet A.

Word

Examples:

- 1101100110101101100 01 is a word over the Binary alphabet B.
- The word *english* is a word over the English alphabet A.
- But *covfefe* as well!

Word

Examples:

- 1101100110101100001 is a word over the Binary alphabet B.
- The word *english* is a word over the English alphabet A.
- But *covfefe* as well!
- Any Java program is a word over the Java alphabet J.

Word

Examples:

- 110110011010110001 is a word over the Binary alphabet B.
- The word *english* is a word over the English alphabet A.
- But *covfefe* as well!
- Any Java program is a word over the Java alphabet J.
- But `0( class +{ public` as well!

Word

Examples:

- 1101100110101100001 is a word over the Binary alphabet B.
- The word *english* is a word over the English alphabet A.
- But *covfefe* as well!
- Any Java program is a word over the Java alphabet J.
- But `0( class +{ public` as well!

Over any alphabet Σ, there is a special word called the *empty word*, written ε: this is the unique sequence of length 0. It contains no letters at all.

Word

Examples:

- 110110011010110001 is a word over the Binary alphabet B.
- The word *english* is a word over the English alphabet A.
- But *covfefe* as well!
- Any Java program is a word over the Java alphabet J.
- But `0( class +{ public` as well!

Over any alphabet Σ, there is a special word called the *empty word*, written ε: this is the unique sequence of length 0. It contains no letters at all.

Remark

Notice how we write letters simply juxtaposed by one another. There is no separator (such as commas) like when we write vector coordinates.

Basic operations on words

Concatenation

Let Σ be an alphabet. Let u and w be words over Σ. Their *concatenation*, written $u \cdot w$ or even more simply uw, is the word obtained by laying out all the letters of w after these of u.

Basic operations on words

Concatenation

Let Σ be an alphabet. Let u and w be words over Σ. Their *concatenation*, written $u \cdot w$ or even more simply uw, is the word obtained by laying out all the letters of w after these of u.

More formally, writing $u = u_1 u_2 \dots u_n$ and $w = w_1 w_2 \dots w_m$ (where all u_i and w_j are letters of Σ), we have:

$$uw = u_1 u_2 \dots u_n w_1 w_2 \dots w_m$$

Basic operations on words

Concatenation

Let Σ be an alphabet. Let u and w be words over Σ. Their *concatenation*, written $u \cdot w$ or even more simply uw, is the word obtained by laying out all the letters of w after these of u.

More formally, writing $u = u_1 u_2 \dots u_n$ and $w = w_1 w_2 \dots w_m$ (where all u_i and w_j are letters of Σ), we have:

$$uw = u_1 u_2 \dots u_n w_1 w_2 \dots w_m$$

The empty word is *neutral* for the concatenation: $w\varepsilon = \varepsilon w = w$ for any word w.

Example

- Over the alphabet B,

$$110 \cdot 0110 = 1100110$$

Example

- Over the alphabet B,

$$110 \cdot 0110 = 1100110$$

- Over the alphabet A,

$$conca \cdot tenation = concatenation$$

Length of a word

Let Σ be an alphabet and w be a word over Σ. The *length* of w, written $|w|$, is the number of letters in w.

Length of a word

Let Σ be an alphabet and w be a word over Σ. The *length* of w, written $|w|$, is the number of letters in w.

- $|\varepsilon| =$

Length of a word

Let Σ be an alphabet and w be a word over Σ. The *length* of w, written $|w|$, is the number of letters in w.

- $|\varepsilon| = 0$

Length of a word

Let Σ be an alphabet and w be a word over Σ. The *length* of w, written $|w|$, is the number of letters in w.

- $|\varepsilon| = 0$
- $|100110101| =$

Length of a word

Let Σ be an alphabet and w be a word over Σ. The *length* of w, written $|w|$, is the number of letters in w.

- $|\varepsilon| = 0$
- $|100110101| = 9$

Length of a word

Let Σ be an alphabet and w be a word over Σ. The *length* of w, written $|w|$, is the number of letters in w.

- $|\varepsilon| = 0$
- $|100110101| = 9$
- $|\texttt{public class Main}| =$

Length of a word

Let Σ be an alphabet and w be a word over Σ. The *length* of w, written $|w|$, is the number of letters in w.

- $|\varepsilon| = 0$
- $|100110101| = 9$
- $|\texttt{public class Main}| = 8$ (when considered over the Java alphabet J)

Language

Language

Let Σ be an alphabet. A *language* over Σ is a set of words over Σ.

Language

Language

Let Σ be an alphabet. A *language* over Σ is a set of words over Σ.

Examples:

Language

Language

Let Σ be an alphabet. A *language* over Σ is a set of words over Σ.

Examples:

- The English language is a language over the English alphabet A.

Language

Language

Let Σ be an alphabet. A *language* over Σ is a set of words over Σ.

Examples:

- The English language is a language over the English alphabet A.
- The set of all binary representations is a language over the binary alphabet B.

Language

Language

Let Σ be an alphabet. A *language* over Σ is a set of words over Σ.

Examples:

- The English language is a language over the English alphabet A.
- The set of all binary representations is a language over the binary alphabet B.
- The set of all valid arithmetic expressions is a language over the calculator alphabet C.

Language

Language

Let Σ be an alphabet. A *language* over Σ is a set of words over Σ.

Examples:

- The English language is a language over the English alphabet A.
- The set of all binary representations is a language over the binary alphabet B.
- The set of all valid arithmetic expressions is a language over the calculator alphabet C.
- The set of all DNA sequences of viruses is a language over the DNA alphabet D.

Language

Language

Let Σ be an alphabet. A *language* over Σ is a set of words over Σ.

Examples:

- The English language is a language over the English alphabet A.
- The set of all binary representations is a language over the binary alphabet B.
- The set of all valid arithmetic expressions is a language over the calculator alphabet C.
- The set of all DNA sequences of viruses is a language over the DNA alphabet D.
- The set of all valid Java programs is a language over the Java alphabet J.

Archetypal examples

For any alphabet Σ:

Archetypal examples

For any alphabet Σ:

- The set Σ itself is a language over Σ. This is the language of all words of length 1 over Σ.

Archetypal examples

For any alphabet Σ:

- The set Σ itself is a language over Σ. This is the language of all words of length 1 over Σ.
- The singleton set $\{\varepsilon\}$ is a language over Σ. This is the language containing only the empty word.

Archetypal examples

For any alphabet Σ:

- The set Σ itself is a language over Σ. This is the language of all words of length 1 over Σ.
- The singleton set $\{\varepsilon\}$ is a language over Σ. This is the language containing only the empty word.
- The empty set $\varnothing$ is a language over Σ. This is the language containing no words at all.

Archetypal examples

For any alphabet Σ:

- The set Σ itself is a language over Σ. This is the language of all words of length 1 over Σ.
- The singleton set $\{\varepsilon\}$ is a language over Σ. This is the language containing only the empty word.
- The empty set $\varnothing$ is a language over Σ. This is the language containing no words at all.

Don't confuse the last two!

Operations on languages

Languages are sets (of words), so we can apply set operations to them: union, intersection, complement.

Operations on languages

Languages are sets (of words), so we can apply set operations to them: union, intersection, complement.

Concatenation

Let Σ be an alphabet. Let K, L be languages over Σ. Their *concatenation*, written $K \cdot L$, is the language containing all words that can be formed by taking a word from K and a word from L and concatenating them.

Operations on languages

Languages are sets (of words), so we can apply set operations to them: union, intersection, complement.

Concatenation

Let Σ be an alphabet. Let K, L be languages over Σ. Their *concatenation*, written $K \cdot L$, is the language containing all words that can be formed by taking a word from K and a word from L and concatenating them.

Formally,

$$K \cdot L = \{u \cdot w \mid u \in K, w \in L\}$$

Operations on languages

Languages are sets (of words), so we can apply set operations to them: union, intersection, complement.

Concatenation

Let Σ be an alphabet. Let K, L be languages over Σ. Their *concatenation*, written $K \cdot L$, is the language containing all words that can be formed by taking a word from K and a word from L and concatenating them.
Formally,

$$K \cdot L = \{u \cdot w \mid u \in K, w \in L\}$$

$K \cdot L$ comprises all the possible combinations of a word of K followed by a word of L.

Power and star

Let Σ be an alphabet. Let L be a language over Σ.

Power and star

Let Σ be an alphabet. Let L be a language over Σ.

The *n-th power* of L, written L^n, is the language containing all words that can be formed by concatenating n words from L.

Power and star

Let Σ be an alphabet. Let L be a language over Σ.

The *n-th power* of L, written L^n, is the language containing all words that can be formed by concatenating n words from L.
Formally,

$$L^n = \underbrace{L \cdot L \cdots L}_{n \text{ times}}$$

Power and star

Let Σ be an alphabet. Let L be a language over Σ.

The *n-th power* of L, written L^n, is the language containing all words that can be formed by concatenating n words from L.
Formally,

$$L^n = \underbrace{L \cdot L \cdots L}_{n \text{ times}}$$

The *Kleene star* of L, written L^*, is the language containing all words that can be formed by concatenating any number of words from L, including the empty word.

Power and star

Let Σ be an alphabet. Let L be a language over Σ.

The *n-th power* of L, written L^n, is the language containing all words that can be formed by concatenating n words from L.
Formally,

$$L^n = \underbrace{L \cdot L \cdots L}_{n \text{ times}}$$

The *Kleene star* of L, written L^*, is the language containing all words that can be formed by concatenating any number of words from L, including the empty word.
Formally,

$$L^* = \bigcup_{n \geq 0} L^n$$

Example

Let Σ be an alphabet.

Example

Let Σ be an alphabet. We have seen that Σ is a language over Σ.

Example

Let Σ be an alphabet. We have seen that Σ is a language over Σ.

What is the language Σ^*?